

Hierarchical Graph Neural Networks for Resilient Intrusion Detection in Consumer IoT With Limited Labeled Data

Guolong Zheng, Changgui Xu, Jianshan Zhang[✉], *Member, IEEE*, Chen Hou, Xu Yang[✉], Tao Huang[✉], Xuan Liu[✉], *Senior Member, IEEE*, and Muhammad Khurram Khan[✉], *Senior Member, IEEE*

Abstract—The growing number of interconnected systems in consumer IoT environments and the increasing intelligence of AI-enabled cyber threats have introduced significant security challenges. Intrusion detection systems must be not only accurate but also resilient to evolving threats and robust against adversarial manipulation. A major limitation is the lack of labeled data for evolving adversarial attacks, which limits the generalization of detection models, making them less practical for the dynamic and heterogeneous nature of IoT networks. To overcome these limitations, we propose *HierGNN*, a hierarchical graph neural network designed for accurate and resilient network intrusion detection using few labeled examples. *HierGNN* utilizes a packet-level graph attention network for adaptive feature filtering, which enhances resilience to subtle data perturbations, coupled with a flow-level temporal model to capture complex spatial and temporal dependencies in traffic patterns. By hierarchically abstracting these features, it learns robust representations of normal and malicious network behavior from limited data. Experimental results on multiple widely used network intrusion datasets demonstrate the *HierGNN*'s remarkable data efficiency and resilience. With as little as 5% of labeled training data, *HierGNN* achieves an F1-score of 0.97 and an accuracy of 0.98 across all the benchmarks. Even in extreme data scarcity, using just 0.1% of labeled data, it maintains a high F1-score of 0.94+ and an accuracy of 0.96+. Furthermore, we demonstrate the model's resilience by evaluating it against common adversarial evasion attacks, such as the Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD). Under these attacks, *HierGNN*'s accuracy only degrades minimally to 0.95, outperforming standard deep learning baselines whose performance drops below 90% under similar conditions. These results validate

HierGNN as a scalable, resilient, and practical method for efficient network intrusion detection with high accuracy and low false positives, even in data-scarce and adversarial IoT environments.

Index Terms—Network intrusion detection, few-shot learning, graph neural networks, hierarchical graph neural network.

I. INTRODUCTION

THE RAPID growing of interconnected systems, particularly within consumer Internet of Things (IoT) environments, has created a hyper-connected ecosystem with 75 billion devices by 2025 [1], dramatically broadening the network attack surface [2]. Consumer IoT devices often prioritize low-cost and rapid market entry over robust security, resulting in systemic weaknesses such as unpatched firmware, weak or default credentials, and insecure communication protocols [2], [3], [4]. Consequently, these vulnerable endpoints can be easily compromised and organized into botnets for large-scale Distributed Denial-of-Service (DDoS) attacks or data exfiltration campaigns, leading to significant financial losses and undermining consumer trust [3], [5].

A primary challenge for modern Network Intrusion Detection Systems (NIDS) in these environments is the dual problem of data scarcity and adversarial vulnerability. Traditional signature-based NIDS are ineffective against novel, zero-day attacks [6]. While supervised machine learning models offer a path to generalization, they typically demand vast quantities of labeled data, a resource that is notoriously scarce in dynamic IoT networks [7]. Furthermore, even well-trained deep learning models are susceptible to adversarial evasion attacks, where malicious actors introduce subtle, carefully crafted perturbations to network traffic to bypass detection [8], [9]. The resilience of a NIDS against such manipulation is therefore as critical as its baseline accuracy [10]. While Graph Neural Networks (GNNs) have emerged as a promising paradigm for modeling complex network traffic [11], many existing GNN-based systems still require substantial labeled data and struggle to filter out irrelevant packet-level noise, which can degrade performance and increase vulnerability to adversarial inputs [12].

Other deep learning architectures have been applied to NIDS, but often operate at a single level of granularity (either packet or flow), neglecting the hierarchical nature of network attacks. For instance, Convolutional Neural Networks (CNNs) can extract spatial features from packets but lack

Received 6 July 2025; revised 13 August 2025; accepted 25 August 2025. Date of publication 29 August 2025; date of current version 25 March 2026. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62402203; in part by the Fujian Provincial Natural Science Foundation under Grant 2024J08278, Grant 2024J08276, and Grant 2024J08277; and in part by the Key Research Project for Young and Middle-aged Researchers by the Fujian Provincial Department of Education under Grant JZ230044; in part by the Science Foundation of Fuzhou under Grant 2025-ZD-023; and in part by the Ongoing Research Funding Program, King Saud University, Riyadh, Saudi Arabia, under Grant ORF-Ctr-2025-4. (Corresponding authors: Tao Huang; Xuan Liu.)

Guolong Zheng, Jianshan Zhang, Chen Hou, Xu Yang, and Tao Huang are with the School of Computer and Data Science, Minjiang University, Fuzhou 350108, China (e-mail: gzheng@mju.edu.cn; jszhang@mju.edu.cn; houchen@mju.edu.cn; xu.yang@mju.edu.cn; huang-tao@mju.edu.cn).

Changgui Xu is with the School of Information Science and Technology, ShanghaiTech University, Shanghai 201210, China (e-mail: xuchg2022@shanghaitech.edu.cn).

Xuan Liu is with the College of Information and Artificial Intelligence (College of Industrial Software), Yangzhou University, Yangzhou 225009, China (e-mail: yusuf@yzu.edu.cn).

Muhammad Khurram Khan is with the Center of Excellence in Information Assurance, DSR, King Saud University, Riyadh 11451, Saudi Arabia (e-mail: mkhurrum@ksu.edu.sa).

Digital Object Identifier 10.1109/TCE.2025.3604228

interpretability [13], while recurrent architectures like Long Short-Term Memory (LSTM) networks model temporal dependencies in flows but may miss coordinated, multi-flow attack patterns [14], [15]. A unified framework that combines multi-level analysis with high data efficiency and adversarial robustness remains a critical challenge in the field [16].

To address these limitations, we propose *HierGNN*, a **two-level hierarchical Graph Neural Network (GNN)** designed for high-accuracy, resilient network intrusion detection using few labeled examples. At the packet level, a Graph Attention Network (GAT) [17] with an adaptive gating mechanism acts as a sophisticated filter, suppressing noisy packet features while amplifying salient indicators of malicious activity. This enhances resilience to the subtle data perturbations characteristic of adversarial attacks [18], [19]. Building upon these refined representations, the second level employs a temporal model with self-attention to model high-level, flow-level interactions over time, capturing the long-range dependencies of sophisticated intrusions [20], [21]. By hierarchically abstracting spatiotemporal features, our model can learn robust representations resilient to adversarial inputs from a limited number of labeled samples.

The efficacy of *HierGNN* is rigorously validated on three widely-used benchmarks (CICIDS2017 [22], ToN-IoT [23], and MedBIoT [24]). With as little as 0.1% of labeled data, *HierGNN* achieves an F1-score of 0.94+ and an accuracy of 0.96+ across all benchmarks. Furthermore, to demonstrate its resilience, we evaluated the model against common adversarial evasion attacks, including the Fast Gradient Sign Method (FGSM) [8] and Projected Gradient Descent (PGD) [9]. Under these attacks, *HierGNN*'s accuracy only degraded to 0.95, outperforming standard deep learning baselines whose performance dropped below 90% under similar conditions. The results validate *HierGNN* as a scalable, resilient, and practical method for network intrusion detection in data-scarce and adversarial consumer IoT environments. The main contributions of this article are:

- We propose a hierarchical graph neural network for NIDS that efficiently utilizes scarce labels. The model learns robust and discriminative flow embeddings by first capturing packet-level spatial dependencies and then abstracting high-level temporal patterns, enabling accurate detection with minimal labeled data.
- We introduce a label-aware learning strategy where an adaptive gating mechanism filters irrelevant packet features. This improves the resilience to noise and allows the model to alleviate data heterogeneity across diverse IoT scenarios.
- Our framework is inherently resilient to adversarial manipulation. The hierarchical feature abstraction process improves robustness, allowing the model to maintain high accuracy against common evasion attacks.
- We conduct extensive experiments on three public NIDS datasets. The results demonstrate that our model significantly outperforms baseline approaches, achieving over 96% accuracy and 0.94 F1-score with as little as 0.1% of labeled data, validating its efficacy in data-scarce and adversarial environments.

The source code for this work is publicly available at <https://github.com/guolong-zheng/HierGNN>.

The remainder of this article is organized as follows: Section II reviews related work. Section III details the architecture of *HierGNN*, while Section IV presents the experimental setup and results. Section V concludes the paper.

II. RELATED WORK

The escalating complexity of cyber threats necessitates advanced Network Intrusion Detection Systems (NIDS) that move beyond static, signature-based methods, which struggle against novel attacks and often cause alert fatigue [25], [26]. Machine learning and deep learning approaches have emerged as a powerful alternative, learning intricate patterns from data to improve detection of previously unseen intrusions [27], [28].

Early ML solutions, such as Support Vector Machines (SVM) and Decision Trees, required extensive manual feature engineering, a time-consuming and expertise-intensive process [29], [30], [31]. Deep learning models, such as Convolutional and Recurrent Neural Networks (CNNs, RNNs), automated this feature extraction from raw traffic [14], [32]. Unsupervised methods like Autoencoders have also been used for anomaly detection [33]. However, these approaches typically process network flows in isolation, failing to capture the rich topological patterns inherent in network traffic interactions [31].

Graph Neural Networks (GNNs) naturally represent network traffic as a graph of interacting entities, making them ideal for capturing the complex relational patterns indicative of sophisticated attacks like APTs [20], [34]. Initial GNN-based NIDS focused on topological structure to detect botnets [35], [36], but often ignored valuable node and edge features. Subsequent research integrated flow statistics with graph structure [37], [38], but these supervised models still required extensive labeled data. To address this, recent works have explored few-shot and self-supervised learning. Methods like Anomal-E [39] and TS-IDS [40] leverage self-supervision to learn from unlabeled data, while FeCoGraph [31] uses a federated contrastive learning approach. While these methods improve data efficiency, they still face challenges with intricate attack patterns and the computational overhead of centralized analysis in large-scale IoT networks [31].

A critical insight is that comprehensive detection requires integrating both fine-grained packet-level details and coarse-grained flow-level patterns [41], [42]. Packet analysis can detect payload-specific attacks, while flow analysis identifies broader patterns like DDoS. Pioneering frameworks like XG-NID [42] and HGNN [43] construct heterogeneous or multi-modal graphs to combine packet and flow data, often using hierarchical GNNs. Despite this progress, a key gap remains in developing a unified framework that combines this hierarchical analysis with a dynamic packet filtering mechanism that can adapt to evolving threats, especially under the constraint of limited labeled data. Our work addresses this gap by proposing a hierarchical GNN that integrates adaptive packet filtering with flow-level temporal analysis for resilient and data-efficient intrusion detection.

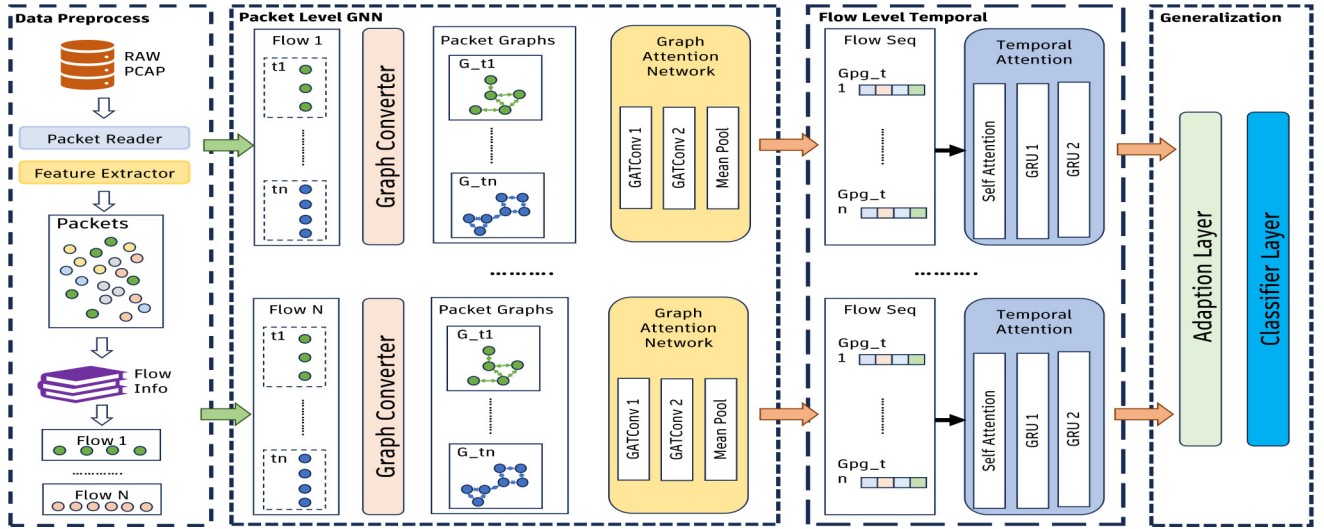


Fig. 1. Architecture of *HierGNN*.

The key difference between *HierGNN* and current techniques, especially FeCoGraph, lies in the mechanism used to achieve robustness and data efficiency. *HierGNN* employs an intra-sample adaptive filtering mechanism. It learns to dynamically weigh and select the most salient features within each individual data sample, effectively filtering out noise and adversarial perturbations on a case-by-case basis. In contrast, methods like FeCoGraph utilize inter-sample label-aware contrastive learning. This approach learns a discriminative embedding space by pulling samples with the same label (e.g., two different benign flows) closer together while pushing samples with different labels (e.g., a benign and a malicious flow) farther apart. While both are powerful techniques, their philosophies are complementary. *HierGNN* focuses on purifying the features of each sample before learning, whereas FeCoGraph focuses on learning robust representations by comparing different samples against each other. Our approach is particularly well-suited to mitigating adversarial attacks that manipulate features within a single sample.

III. METHODOLOGY

As illustrated in Fig. 1, the proposed Hierarchical GNN (*HierGNN*) architecture is composed of three primary modules following the initial data preprocessing that transform raw PCAP packets into graphs. First, the Packet-Level GNN module processes individual network flows by converting packet sequences into a series of dynamic packet graphs. Each graph is then passed through a Graph Attention Network, utilizing two GATConv layers and a mean pooling layer, to generate a contextualized graph embedding.

Second, the Flow-Level Temporal module receives the sequence of these embeddings and employs a temporal attention mechanism, which includes self-attention and several GRU layers, to capture long-range temporal dependencies within the flow.

Finally, the Generalization module achieves robust intrusion detection by employing an adaptation layer that integrates

a Model-Agnostic Meta-Learning (MAML) framework for learning from limited data and an adaptive attention mechanism to bolster adversarial resilience, before passing the resulting flow representation to a final classifier. This hierarchical design enables *HierGNN* to learn both fine-grained packet interactions and long-term behavioral patterns, effectively addressing the multi-scale nature of network attacks.

A. Data Preprocessing

HierGNN is designed to operate directly on packet-level information to dynamically learn flow characteristics. Therefore, all features are extracted exclusively from packet headers, without relying on Deep Packet Inspection or any payload content. The flow-level CSV files provided with the datasets are used solely for ground-truth labeling and for grouping packets into their respective flows based on their 5-tuple (source/destination IP, source/destination port, protocol) and timestamps. No pre-computed, high-level flow features from these files are used.

The initial stage of our methodology involves a comprehensive data preprocessing pipeline designed to convert raw network traffic captures into a structured, graph-based format amenable to the GNN model. The pipeline takes pcap files as primary input. For labeled datasets, it additionally ingests corresponding csv files to map network flows to their ground-truth labels (e.g., benign or a specific attack type), which are then converted to binary class labels.

The pipeline begins by grouping packets from the input pcap file into bidirectional flows using a normalized five-tuple key. Flow and packet selection are governed by two temporal constraints: first, an inactivity timeout of 300 seconds defines the maximum idle time before a new flow is initiated for subsequent packets, effectively segmenting long-lived connections. Second, we define a variable-size packet window, considering the first n packets of any given

flow segment for feature extraction, where n is capped at a maximum of 20.¹

This is represented as packets $t_1, \dots, t_n$ for each flow in Fig. 1. This dual-window approach ensures that the analysis focuses on the crucial initial phase of network connections while maintaining computational tractability. For each packet within this window, a 10-dimensional feature vector is meticulously extracted to capture its essential characteristics, providing a detailed snapshot of each packet's role and context within its flow.

A key innovation in our preprocessing is the representation of each network flow as a temporal sequence of graphs. Instead of constructing a single, static graph for an entire flow, we model each individual packet as a distinct graph object using the PyTorch Geometric library. Each “packet-graph” is a small, fully-connected graph comprising three nodes. The 10-dimensional feature vector of the packet is duplicated and assigned as the feature set for each of these three nodes.² This design transforms a flow of N packets into a sequence of N graphs, $G_{t_1}, G_{t_2}, \dots, G_{t_n}$, which allows the packet-level GNN module to learn fine-grained intra-packet feature interactions before the flow-level module captures their temporal evolution.

Following feature extraction, all packet feature vectors across the entire training dataset are normalized using a StandardScaler to have zero mean and unit variance. The parameters of this scaler are saved and applied consistently to any validation or testing data to prevent information leakage. Finally, the complete processed dataset containing the sequences of packet-graphs. This creates a compact and efficient data structure that can be loaded directly into memory for model training and evaluation.

B. Packet-Level Processing: Dynamic Feature Filtering and Graph Attention

The packet-level processing module transforms raw network packet features, $\mathbf{x}_i$ for each packet i , into contextualized packet graph embeddings. This is achieved through a two-step process: dynamic feature filtering followed by graph attention-based aggregation.

1) *Dynamic Feature Filtering*: Initially, each packet's feature vector $\mathbf{x}_i \in \mathbb{R}^{D_{in}}$, where D_{in} is the input feature dimension, undergoes a dynamic filtering process designed to isolate the most salient features. This process consists of two parallel components:

- 1) *Feature Importance Scoring and Masking*: A feature importance network, MLP_{imp} , computes an importance score vector $\mathbf{s}_i^{imp} \in [0, 1]^{D_{in}}$. This network is a three-layer Multi-Layer Perceptron (MLP) with Leaky Rectified Linear Unit (LeakyReLU) activations, concluding with a Sigmoid function to normalize scores between 0 and 1.

$$\mathbf{s}_i^{imp} = \text{Sigmoid}(\text{MLP}_{imp}(\mathbf{x}_i)) \quad (1)$$

A dynamic threshold τ_i is determined using a predefined quantile q_r (termed `filter_ratio`) of these scores. This threshold is used to generate a binary mask $\mathbf{m}_i^{thresh}$ that retains features with scores in the top $(1 - q_r)$ percentile.

$$\tau_i = \text{quantile}(\mathbf{s}_i^{imp}, 1 - q_r) \quad (2)$$

$$\mathbf{m}_i^{thresh} = \mathbb{I}(\mathbf{s}_i^{imp} \geq \tau_i) \quad (3)$$

where $\mathbb{I}(\cdot)$ is the indicator function.

- 2) *Gating Mechanism*: Concurrently, a gating network, MLP_{gate} , computes a gating score vector $\mathbf{g}_i^{gate} \in [0, 1]^{D_{in}}$. This network is a two-layer MLP employing a hyperbolic tangent (Tanh) activation in its hidden layer and a final Sigmoid activation.

$$\mathbf{g}_i^{gate} = \text{Sigmoid}(\text{MLP}_{gate}(\mathbf{x}_i)) \quad (4)$$

The original features $\mathbf{x}_i$ are then modulated through element-wise multiplication with both the binary mask and the gating scores to produce the filtered feature vector $\mathbf{x}'_i$.

$$\mathbf{x}'_i = \mathbf{x}_i \odot \mathbf{m}_i^{thresh} \odot \mathbf{g}_i^{gate} \quad (5)$$

where $\odot$ denotes the Hadamard product. This vector $\mathbf{x}'_i$ represents the dynamically refined packet features fed into the subsequent stage.

2) *Graph Attention Network*: The filtered packet features $\mathbf{X}' = \{\mathbf{x}'_1, \dots, \mathbf{x}'_N\}$ and their corresponding communication links, represented by an adjacency matrix $\mathcal{A}$, constitute a packet graph. This graph is processed by a two-layer Graph Attention Network (GAT).

The first GAT layer, GATConv_1 , computes hidden representations using K_h attention heads. The output is passed through an Exponential Linear Unit (ELU) activation and regularized with dropout.

$$\mathbf{H}^{(1)} = \text{Dropout}(\text{ELU}(\text{GATConv}_1(\mathbf{X}', \mathcal{A}; \Theta_{gat1}, K_h))) \quad (6)$$

The second GAT layer, GATConv_2 , takes the hidden representations $\mathbf{H}^{(1)}$ as input, using a single attention head to consolidate features. The output is then stabilized using Layer Normalization to produce the final node embeddings $\mathbf{H}^{(2)}$.

$$\mathbf{H}^{(2)} = \text{LayerNorm}(\text{GATConv}_2(\mathbf{H}^{(1)}, \mathcal{A}; \Theta_{gat2}, 1)) \quad (7)$$

The attention coefficient α_{ij} between a node i and its neighbor $j \in \mathcal{N}_i$ within each GAT layer is computed as:

$$e_{ij} = \text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]) \quad (8)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})} \quad (9)$$

¹The 20 is selected based on an empirical trade-off between capturing essential information and maintaining computational efficiency. The initial packets of a network flow are often the most informative for intrusion detection, as they typically contain handshake protocols, session initiations, and initial data exchanges that can reveal the flow's intent. Furthermore, the flow with more than 20 packets is then split into multiple flows, and information between them is then captured at flow level.

²The 3-node fully-connected graph architecture is designed to enable the Graph Attention Network (GAT) to perform intra-packet feature learning. The GAT's self-attention mechanism can weigh and learn the interactions and relative importance among the different features within that single packet. This creates a richer, more contextualized packet embedding before the temporal model analyzes the sequence of packets. It allows the model to discover which feature combinations (e.g., a specific flag set with a particular port) are most indicative of malicious activity at the most granular level.

where $\mathbf{h}_i$ is the feature vector for node i , $\mathbf{W}$ is a shared learnable linear transformation, and $\mathbf{a}$ is a learnable attention weight vector.

Finally, a single graph-level embedding $\mathbf{g}_{pg}$ for the entire packet graph is derived by applying a global mean pooling operation to the final node embeddings.

$$\mathbf{g}_{pg} = \text{GlobalMeanPool}(\mathbf{H}^{(2)}) \quad (10)$$

This embedding, $\mathbf{g}_{pg}$, encapsulates the processed information of a single packet graph, corresponding to a snapshot of network activity.

This two-part filtering mechanism is intentionally sophisticated to enhance adversarial resilience. The importance network combined with quantile masking performs a hard selection, forcing the model to ignore irrelevant features and reducing the overall attack surface. The gating network then performs a soft re-weighting of the remaining features, providing a more nuanced, data-dependent modulation that can suppress features that may be subtly perturbed. This combination of coarse-grained filtering and fine-grained gating creates a robust defense that is difficult for an adversary to predict and evade.

C. Flow-Level Temporal Modeling

The flow-level module processes sequences of packet graph embeddings $\mathbf{S}_{flow} = (\mathbf{g}_{pg,1}, \mathbf{g}_{pg,2}, \dots, \mathbf{g}_{pg,T})$ derived from the packet-level stage. This module is designed to capture temporal dependencies within a network flow.

1) *Temporal Self-Attention*: A multi-head self-attention mechanism is first applied to the sequence $\mathbf{S}_{flow}$ to dynamically weigh the importance of different packet graph embeddings across the flow's timeline. This allows the model to focus on the most relevant temporal steps.

$$\mathbf{A}_{flow} = \text{MultiheadAttention}(\mathbf{S}_{flow}, \mathbf{S}_{flow}, \mathbf{S}_{flow}, \text{mask}) \quad (11)$$

where $\mathbf{A}_{flow} = (\mathbf{a}_1, \dots, \mathbf{a}_T)$ are the attention-weighted representations. A key padding mask is utilized to ensure that padded elements in variable-length flow sequences do not contribute to the attention computation.

2) *Bidirectional Gated Recurrent Unit*: The sequence of attended features $\mathbf{A}_{flow}$ is then processed by a multi-layer bidirectional GRU. To efficiently handle the variable lengths of flows, the sequences are packed using `torch.nn.utils.rnn.pack_padded_sequence` before being fed into the GRU, which allows the model to operate only on the valid, unpadded time steps.

$$\mathbf{H}^{gru} = \text{BiGRU}(\text{pack_padded}(\mathbf{A}_{flow})) \quad (12)$$

The output, $\mathbf{H}^{gru} = (\mathbf{h}_1^{gru}, \dots, \mathbf{h}_T^{gru})$, consists of the concatenated forward and backward hidden states, capturing both past and future context at each time step.

3) *Temporal Fusion and Aggregation*: The BiGRU output sequence $\mathbf{H}^{gru}$ is passed through a temporal fusion layer to refine the features. This layer consists of a linear transformation, a Tanh activation, dropout, and Layer Normalization.

$$\mathbf{h}_t^{fused} = \text{LayerNorm}(\text{Dropout}(\text{Tanh}(\text{MLP}_{ff}(\mathbf{h}_t^{gru})))) \quad (13)$$

To produce a single representative vector for the entire flow, the fused temporal features are aggregated. First, the mean of the valid (unpadded) feature vectors $\{\mathbf{h}_t^{fused}\}$ is computed. This mean vector is then passed through a final two-layer aggregation MLP, MLP_{agg} , which uses ReLU and Tanh activations.

$$\bar{\mathbf{h}}^{fused} = \frac{1}{T_{valid}} \sum_{t=1}^{T_{valid}} \mathbf{h}_t^{fused} \quad (14)$$

$$\mathbf{f}_{agg} = \text{Tanh}(\text{MLP}_{agg}(\bar{\mathbf{h}}^{fused})) \quad (15)$$

The resulting vector $\mathbf{f}_{agg}$ is the final, comprehensive representation of the network flow.

This BiGRU architecture is chosen over a full Transformer architecture due to its computational efficiency on the relatively short sequences of packets found in network flows. For such sequences, a GRU-based model provides a strong balance of performance and lower computational overhead, making it more suitable for potential edge deployment.

D. Generalization Module

The Generalization Module receives the aggregated flow representation $\mathbf{f}_{agg}$ and applies a Model-Agnostic Meta-Learning (MAML) framework that operates on adversarially-adapted feature representations before passing to a final classifier. MAML is chosen due to its flexibility and suitability for the NIDS task. It learns a model initialization that can be rapidly adapted to new tasks with few gradient steps. Especially in the context of intrusion detection, which requires adapting the model's decision boundary, not just learning a static feature space, MAML's approach of finding an optimal starting point for fine-tuning is better suited for adapting the detection logic itself, making it more effective for generalizing to unseen threats from a small number of examples.

1) *Adversarial Resilience via Adaptive Attention*: To improve the model's robustness against adversarial attacks with subtle perturbations, we first process $\mathbf{f}_{agg}$ through an adaptive attention layer. This layer learns to dynamically re-weight the feature vector, emphasizing salient features while suppressing those likely manipulated by an adversary. An attention network, MLP_{adapt} , computes a vector of importance scores, which are normalized via a Softmax function to produce an attention mask $\mathbf{a}_{adapt}$.

$$\mathbf{a}_{adapt} = \text{Softmax}(\text{MLP}_{adapt}(\mathbf{f}_{agg})) \quad (16)$$

This mask is then applied to the original representation via element-wise multiplication to yield a resilient flow vector $\mathbf{f}_{resilient}$.

$$\mathbf{f}_{resilient} = \mathbf{f}_{agg} \odot \mathbf{a}_{adapt} \quad (17)$$

The refined vector serves as the input for the subsequent meta-learning stage, ensuring that the model learns from a more robust and stable feature space.

2) *Few-Shot Generalization With MAML*: To address the challenge of data scarcity, we employ the Model-Agnostic Meta-Learning (MAML) algorithm [44]. MAML trains the model's initial parameters, denoted by θ to be a suitable initialization that can be rapidly adapted to a new task using only a few labeled examples. The meta-training process involves a two-step update. First, for each task T_i , an inner update adapts the parameters to θ'_i using a small “support set” S_i of labeled examples:

$$\theta'_i = \theta - \alpha \nabla \theta \mathcal{L}_{T_i}(\theta, S_i) \quad (18)$$

Second, an outer update optimizes the initial parameters θ by evaluating the adapted model on a query set Q_i and minimizing the loss across all tasks:

$$\theta \leftarrow \theta - \beta \nabla \theta \sum T_i \mathcal{L}_{T_i}(\theta'_i, Q_i) \quad (19)$$

where α and β are the inner and outer loop learning rates, respectively.

This meta-learning process trains an initial model state for rapid and effective generalization, making *HierGNN* highly data-efficient.

3) *Final Classification*: The final component is a classification head, a simple MLP denoted as $\text{MLP}_{\text{class}}$, which takes the resilient flow representation $f_{\text{resilient}}$ and maps it to a final prediction. During inference or fine-tuning on a specific task, the adapted parameters θ_i are used. The final output is passed through a Sigmoid function to produce a probability score $\hat{y} \in [0, 1]$, indicating whether the network flow is malicious.

$$\hat{y} = \text{Sigmoid}(\text{MLP}_{\text{class}}(f_{\text{resilient}}; \theta'_i)) \quad (20)$$

E. Model Interpretability

A significant challenge for complex deep learning models in security is their lack of transparency. However, *HierGNN*'s architecture includes components that provide a degree of explainability. The feature importance scores (s_i^{imp}) from the packet-level filtering module allow an analyst to identify which packet features the model is focusing on. Similarly, the temporal self-attention weights from the flow-level module highlight which packets in a sequence are most critical to the model's final decision. This hierarchical interpretability can help an analyst understand not only what is classified as an attack, but also why, by pinpointing both the suspicious features and the key temporal moments within a network flow.

IV. EXPERIMENTAL EVALUATION

This section presents a comprehensive evaluation of *HierGNN* on multiple real-world Network Intrusion Detection datasets. The experiments are designed to validate that *HierGNN* achieves high-accuracy intrusion detection with high data efficiency, even in scenarios of extreme data scarcity and against adversarial attacks. To systematically assess *HierGNN*, we propose the following research questions:

- *RQ1: Effectiveness in General Scenarios*. How does *HierGNN*'s detection performance compare against state-of-the-art methods in standard settings where labeled training data is not so limited (e.g., 10%)?
- *RQ2: Robustness to Extreme Data Scarcity*. How robust is *HierGNN*'s performance when labeled data is extremely scarce (e.g., 500), and how does it compare to baseline methods under such constraints?
- *RQ3: Resilience against Adversarial Attacks*. To what extent does *HierGNN* maintain its detection accuracy when subjected to adversarial evasion attacks (e.g., FGSM and PGD)?
- *RQ4: Architectural Contribution (Ablation Study)*. What is the contribution of *HierGNN*'s key architectural components—the packet-level adaptive filtering and the flow-level temporal GNN—to its overall performance and data efficiency?
- *RQ5: Computational Performance and Edge Feasibility*. How efficient is *HierGNN* in terms of computational overhead (e.g., inference latency, memory footprint), and is it practical for deployment in resource-constrained edge environments?

A. Experiment Setup

1) *Datasets*: We evaluate our model, *HierGNN*, on three widely used network intrusion datasets: CICIDS2017 [22], MedBIoT [45], and ToN-IoT [23]. These datasets were chosen to represent diverse intrusion scenarios with varied background traffic and attack methodologies. All datasets provide both packet-level information in pcap format and flow-level labels in csv format. Each dataset is described as follows:

- *CICIDS2017* [22]: This dataset contains over 2.8 million flow instances, with 80.3% benign and 19.7% malicious traffic. The data was captured over a five-day period and includes a variety of modern attacks such as DoS/DDoS, Brute Force, Heartbleed, Botnet, and Web attacks.
- *MedBIoT* [45]: A dataset focused on IoT botnets within a medium-sized IoT network. It comprises approximately 17.8 million network packets, where 70% represents legitimate traffic and 30% constitutes malicious botnet activity.
- *ToN-IoT* [23]: Generated from a testbed simulating realistic IoT and Industry 4.0 networks, this dataset contains 16.9 million records, with 36% benign traffic and 64% attack traffic. It is designed to reflect the complexity and scale of modern smart environments.

While comprehensive, these datasets are static and may not fully capture the dynamic nature and “concept drift” of a live production network. The specific traffic profiles may also differ from a target deployment environment. Consequently, while our results demonstrate *HierGNN*'s strong potential, real-world performance will invariably depend on the similarity between the training data and the live traffic. However, we argue that *HierGNN*'s design, particularly its generalization module, makes it inherently well-suited to address this challenge. It is designed for rapid adaptation, allowing it to be efficiently fine-tuned with a small number of labeled samples from a new environment to maintain high accuracy.

2) *Data Preparation*: To comprehensively evaluate our model's performance under varying conditions of data availability, we employ two data splitting settings:

- 1) *Percentage-Based Splitting*: In this setting, we partition the dataset into training and testing sets based on a fixed ratio. We create training sets by randomly sampling $\{0.1\%, 1\%, 5\%, 10\%, 30\%, 50\%, 70\%\}$ of the total data. The remaining data in each case forms the corresponding test set (i.e., $\{99.9\%, 99\%, \dots, 30\%\}$). This allows us to systematically measure how the model's detection accuracy scales with the volume of available training data.
- 2) *Fixed-Size Sampling*: This methodology evaluates the model's effectiveness in a low-data regime. We randomly sample a total of N instances from the entire dataset, where N is varied from $\{300, 500, 700, 1,000\}$. This approach simulates a realistic scenario where a small, collected dataset may not contain uniform representations of all classes, thereby testing the model's performance and robustness against class imbalance.
- 3) *Baseline Methods*: We conduct a comprehensive performance comparison of *HierGNN* against two main categories of baselines: a suite of traditional machine learning models and state-of-the-art GNN-based approaches. The selected traditional models, chosen for their prevalence in NIDS literature, include Decision Tree (DT), Random Forest (RF), Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and XGBoost.

Our selection of GNN baselines prioritized methods with publicly available source code to ensure that our comparisons are both reproducible and fair. We compare our model against two prominent flow-level GNNs for which official implementations exist: E-GraphSAGE [38], a GNN model that leverages edge features for classification, and AnomalE [39], a self-supervised GNN for anomaly detection. Several related methods, like FeCoGraph [31], are not compared as we could not find public available code, re-implementing those complicated methods can introduce subtle deviations that may not reflect their true performance, leading to potentially unfair evaluations.

4) *Evaluation Metrics*: To provide a comprehensive and robust assessment of our model's performance, we employ a set of standard and widely used classification metrics. These metrics are derived from the four primary outcomes of a binary classification task: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). In the context of intrusion detection, these are defined as:

- *True Positive (TP)*: An attack instance correctly identified as an attack.
- *True Negative (TN)*: A benign instance correctly identified as benign.
- *False Positive (FP)*: A benign instance incorrectly identified as an attack (a false alarm).
- *False Negative (FN)*: An attack instance incorrectly identified as benign (a missed detection).

The following metrics are used for evaluation:

Accuracy measures the overall proportion of correctly classified instances.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (21)$$

Precision (or Positive Predictive Value) measures the proportion of predicted attacks that were actual attacks. High precision is critical for minimizing the operational cost of investigating false alarms.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (22)$$

Recall (or True Positive Rate) measures the proportion of actual attack instances that were correctly identified. High recall is essential for ensuring that threats are not missed.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (23)$$

F1-Score is the harmonic mean of Precision and Recall, providing a single score that balances the trade-off between false positives and false negatives.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (24)$$

5) *Experiment Setup*: All experiments were implemented in Python 3.8.8 using the PyTorch 1.13.0 framework. For the GNN-based baselines, PyTorch Geometric 2.3.0 and DGL 1.2 were also utilized. The experiments were conducted on a server running Ubuntu 22.04, equipped with one Intel Xeon Silver 4310 CPU of 2.10GHz, and two NVIDIA GeForce RTX 4090 GPUs.

All experiments were conducted multiple times with different random seeds, and the reported results were stable and consistent across these runs. This confirms the reliability of our findings.

B. Experiment Result

1) *RQ1: Effectiveness in General Scarcity Scenarios*: To answer the first research question on the effectiveness of *HierGNN* in typical percentage-based environments, we conducted a performance comparison against state-of-the-art baselines. We evaluated all models on the MedBioT, ToN-IoT, and CICIDS2017 datasets, training them with limited subsets of the available labeled data: 10%, 1%, 0.5% and 1%. This experimental design directly tests the models' ability to generalize from sparse supervision, a common challenge in real-world IoT deployments. The results are detailed in Table I.

The results across all three datasets consistently demonstrate the superior performance of *HierGNN* compared to the baseline methods, even when trained on a very small fraction of the data.

On the **MedBioT dataset**, *HierGNN* achieves an accuracy of 98.15% with only 0.1% of the training data, outperforming the next best model, XGBoost, by a significant margin of 5.05%. As the percentage of training data increases to 10%, *HierGNN*'s accuracy reaches 99.15%, maintaining a substantial lead over all other methods. A similar trend is observed for Precision, Recall, and F1-Score, where *HierGNN* consistently achieves scores above 97.90% across all training data percentages, highlighting its robustness and effectiveness.

For the **ToN-IoT dataset**, *HierGNN* again shows remarkable performance. With just 0.1% of the data, it records an

TABLE I
PERFORMANCE COMPARISON ON MEDBIOT, ToN-IoT, AND CICIDS2017 DATASETS. THE COLUMNS LABELED 0.1%, 0.5%, 1%, AND 10% INDICATE THE PERCENTAGE OF THE FULL TRAINING SET USED FOR MODEL TRAINING

Dataset	Method	Accuracy (%)				Precision (%)				Recall (%)				F1-Score (%)			
		0.1%	0.5%	1%	10%	0.1%	0.5%	1%	10%	0.1%	0.5%	1%	10%	0.1%	0.5%	1%	10%
Med BioT	DT	90.25	91.15	91.85	92.55	88.10	89.00	89.70	90.40	88.50	89.30	90.00	90.70	88.30	89.15	89.85	90.55
	SVM	89.10	90.00	90.70	91.40	87.20	88.10	88.80	89.50	87.80	88.60	89.30	90.00	87.50	88.35	89.05	89.75
	KNN	88.50	89.40	90.10	90.80	86.50	87.40	88.10	88.80	87.10	87.90	88.60	89.30	86.80	87.65	88.35	89.05
	XGBoost	93.10	93.80	94.30	94.95	91.90	92.60	93.10	93.75	92.30	93.00	93.50	94.15	92.10	92.80	93.30	93.95
	E-GraphSAGE	92.60	93.30	93.80	94.45	91.30	92.00	92.50	93.15	91.70	92.40	92.90	93.55	91.50	92.20	92.70	93.35
	Anomal-E	91.70	92.40	92.90	93.55	90.50	91.20	91.70	92.35	90.90	91.60	92.10	92.75	90.70	91.40	91.90	92.55
	HierGNN	98.15	98.55	98.85	99.15	97.90	98.30	98.60	98.90	98.05	98.45	98.75	99.05	97.97	98.37	98.67	98.97
ToN-IoT	DT	88.10	89.00	89.70	90.40	87.50	88.40	89.10	89.80	87.80	88.60	89.30	90.00	87.65	88.50	89.20	89.90
	SVM	86.20	87.10	87.80	88.50	85.80	86.70	87.40	88.10	86.00	86.90	87.60	88.30	85.90	86.80	87.50	88.20
	KNN	85.10	86.00	86.70	87.40	84.90	85.80	86.50	87.20	85.00	85.90	86.60	87.30	84.95	85.85	86.55	87.25
	XGBoost	88.90	89.80	90.50	91.20	88.20	89.10	89.80	90.50	88.50	89.40	90.10	90.80	88.35	89.25	89.95	90.65
	E-GraphSAGE	84.50	85.40	86.10	86.80	84.10	85.00	85.70	86.40	84.30	85.20	85.90	86.60	84.20	85.10	85.80	86.50
	Anomal-E	87.80	88.70	89.40	90.10	87.20	88.10	88.80	89.50	87.50	88.40	89.10	89.80	87.35	88.25	88.95	89.65
	HierGNN	95.40	95.70	95.90	96.10	95.30	95.60	95.80	96.00	95.30	95.60	95.80	96.10	95.30	95.60	95.80	96.05
CICIDS 2017	DT	90.15	91.05	91.75	92.45	88.70	89.60	90.30	91.00	89.10	89.90	90.60	91.30	88.90	89.75	90.45	91.15
	SVM	89.20	90.10	90.80	91.50	87.50	88.40	89.10	89.80	88.00	88.80	89.50	90.20	87.75	88.60	89.30	90.00
	KNN	82.50	83.40	84.10	84.80	46.20	46.90	47.50	48.10	41.50	42.20	42.80	43.40	43.70	44.45	45.10	45.65
	XGBoost	90.80	91.70	92.40	93.10	89.80	90.70	91.40	92.10	90.20	91.00	91.70	92.40	90.00	90.85	91.55	92.25
	E-GraphSAGE	86.10	87.00	87.70	88.40	85.20	86.10	86.80	87.50	64.50	65.30	66.00	66.70	73.50	74.40	75.15	75.85
	Anomal-E	85.40	86.30	87.00	87.70	80.10	81.00	81.70	82.40	66.80	67.60	68.30	69.00	72.80	73.70	74.45	75.15
	HierGNN	95.25	96.15	97.25	98.25	95.20	96.10	97.20	98.10	95.10	96.00	97.10	98.20	95.15	96.05	97.15	98.15

accuracy of 95.40%, which is 6.5% higher than the second-best performer, XGBoost. The performance gap remains significant as more data is used, with *HierGNN* reaching 96.10% accuracy at the 10% level. The F1-Score for *HierGNN* starts at a high of 95.30% and improves to 96.05%, whereas other methods struggle to get past 90%. This indicates *HierGNN*'s strong ability to balance precision and recall effectively, even with limited labeled examples.

The **CICIDS2017 dataset** is known for its complexity and class imbalance, which is reflected in the lower performance of some baseline models like KNN and E-GraphSAGE, especially in terms of F1-score. Despite this, *HierGNN* demonstrates exceptional performance. It starts with an accuracy of 95.25% at the 0.1% level, surpassing the next best, XGBoost, by 4.45%. When trained with 10% of the data, its accuracy climbs to 98.25%. Most notably, its F1-Score remains consistently high and balanced (95.15% to 98.15%), in stark contrast to methods like KNN and E-GraphSAGE whose F1-scores indicate a significant drop in performance due to poor recall.

In conclusion, the experimental results unequivocally show that *HierGNN* is highly effective in scenarios with limited labeled data. Across all tested datasets and all evaluated percentages of training data, *HierGNN* consistently and significantly outperforms state-of-the-art methods in accuracy, precision, recall, and F1-score. Its ability to achieve high performance with as little as 0.1% of the training data demonstrates its powerful learning capability and its potential for real-world applications where obtaining large amounts of labeled data is often impractical and expensive.

2) **RQ2: Robustness to Extreme Data Scarcity:** To address RQ2, we push the boundaries of data scarcity further, evaluating model performance under extreme conditions where labeled data is exceptionally rare. We trained all models on all three datasets using only 300, 500, 700 and 1000 labeled training instances. This experiment simulates a worst-case

scenario for model training, testing the absolute limits of each model's learning capability. The comprehensive results are presented in Table II.

The results from the data scarcity experiments further underscore the robustness of *HierGNN*. Even with an extremely limited number of training samples, *HierGNN* maintains a commanding lead over all other evaluated methods.

On the **MedBioT dataset**, with only 300 training records, *HierGNN* achieves an F1-Score of 97.27%. This is a substantial 5.87% higher than the next-best model, XGBoost. This performance gap remains consistent as the number of samples increases to 1000, where *HierGNN* reaches a 97.92% F1-Score. This demonstrates that *HierGNN* can build a highly effective detection model from a minimal data footprint.

The performance on the **ToN-IoT dataset** tells a similar story. *HierGNN* starts with a 94.20% accuracy with 300 samples, while the runner-up, XGBoost, only achieves 88.05%. The F1-Score of *HierGNN* is particularly notable, starting at 94.10% and increasing to 95.20%, while all other methods remain below 88.20%. This highlights *HierGNN*'s superior learning efficiency in data-starved environments.

On the complex **CICIDS2017 dataset**, the strength of *HierGNN* is even more apparent. While most baseline methods struggle, *HierGNN* achieves an F1-Score of 94.05% with just 300 samples. In contrast, models like KNN and E-GraphSAGE show a severe degradation in performance, with F1-scores of 43.10% and 72.80% respectively, indicating their inability to handle the dataset's complexity with scarce data. *HierGNN*'s performance remains stable and high, reinforcing its suitability for challenging real-world scenarios.

In response to RQ2, the evidence strongly supports that *HierGNN* is exceptionally robust to conditions of extreme data scarcity. Across all datasets, its performance in terms of accuracy, precision, recall, and F1-score is significantly higher than state-of-the-art methods, even when trained with as few as 300 records. This resilience makes *HierGNN* an ideal

TABLE II
PERFORMANCE COMPARISON ON MEDBIOT, ToN-IoT, AND CICIDS2017 DATASETS. THE COLUMNS LABELED 300, 500, 700, AND 1000 INDICATE THE NUMBER OF RECORDS FROM THE FULL TRAINING SET USED FOR MODEL TRAINING

Dataset	Method	Accuracy (%)				Precision (%)				Recall (%)				F1-Score (%)			
		300	500	700	1000	300	500	700	1000	300	500	700	1000	300	500	700	1000
Med BioT	DT	90.05	90.55	90.95	91.25	87.90	88.40	88.80	89.10	88.30	88.80	89.20	89.50	88.10	88.60	89.00	89.30
	SVM	88.90	89.40	89.80	90.10	87.00	87.50	87.90	88.20	87.60	88.10	88.50	88.80	87.30	87.80	88.20	88.50
	KNN	88.30	88.80	89.20	89.50	86.30	86.80	87.20	87.50	86.90	87.40	87.80	88.10	86.60	87.10	87.50	87.80
	XGBoost	92.40	92.80	93.10	93.45	91.20	91.60	91.90	92.25	91.60	92.00	92.30	92.65	91.40	91.80	92.10	92.45
	E-GraphSAGE	91.90	92.30	92.60	92.95	90.60	91.00	91.30	91.65	91.00	91.40	91.70	92.05	90.80	91.20	91.50	91.85
	Anomal-E	91.00	91.40	91.70	92.05	89.80	90.20	90.50	90.85	90.20	90.60	90.90	91.25	90.00	90.40	90.70	91.05
	<i>HierGNN</i>	97.45	97.75	97.95	98.10	97.20	97.50	97.70	97.85	97.35	97.65	97.85	98.00	97.27	97.57	97.77	97.92
ToN-IoT	DT	87.20	87.50	87.70	87.90	86.65	86.95	87.15	87.35	86.95	87.25	87.45	87.65	86.80	87.10	87.30	87.50
	SVM	85.35	85.65	85.85	86.05	84.95	85.25	85.45	85.65	85.15	85.45	85.65	85.85	85.05	85.35	85.55	85.75
	KNN	84.25	84.55	84.75	84.95	84.05	84.35	84.55	84.75	84.15	84.45	84.65	84.85	84.10	84.40	84.60	84.80
	XGBoost	88.05	88.35	88.55	88.75	87.35	87.65	87.85	88.05	87.65	87.95	88.15	88.35	87.50	87.80	88.00	88.20
	E-GraphSAGE	83.65	83.95	84.15	84.35	83.25	83.55	83.75	83.95	83.45	83.75	83.95	84.15	83.35	83.65	83.85	84.05
	Anomal-E	86.95	87.25	87.45	87.65	86.35	86.65	86.85	87.05	86.65	86.95	87.15	87.35	86.50	86.80	87.00	87.20
	<i>HierGNN</i>	94.20	94.60	95.00	95.30	94.10	94.50	94.90	95.20	94.10	94.50	94.90	95.20	94.10	94.50	94.90	95.20
CICIDS 2017	DT	89.50	89.90	90.20	90.50	88.10	88.50	88.80	89.10	88.50	88.90	89.20	89.50	88.30	88.70	89.00	89.30
	SVM	88.60	89.00	89.30	89.60	86.90	87.30	87.60	87.90	87.40	87.80	88.10	88.40	87.15	87.55	87.85	88.15
	KNN	81.90	82.30	82.60	82.90	45.60	46.00	46.30	46.60	40.90	41.30	41.60	41.90	43.10	43.50	43.80	44.10
	XGBoost	90.20	90.60	90.90	91.20	89.20	89.60	89.90	90.20	89.60	90.00	90.30	90.60	89.40	89.80	90.10	90.40
	E-GraphSAGE	85.50	85.90	86.20	86.50	84.60	85.00	85.30	85.60	63.90	64.30	64.60	64.90	72.80	73.20	73.50	73.80
	Anomal-E	84.80	85.20	85.50	85.80	79.50	79.90	80.20	80.50	66.20	66.60	66.90	67.20	72.20	72.60	72.90	73.20
	<i>HierGNN</i>	94.15	94.55	94.85	95.10	94.10	94.50	94.80	95.05	94.00	94.40	94.70	94.95	94.05	94.45	94.75	95.00

TABLE III
PERFORMANCE COMPARISON ON MEDBIOT, ToN-IoT, AND CICIDS2017 (TRAINED ON 1000 LABELED DATA RECORDS) BEFORE AND AFTER ADVERSARIAL EVASION ATTACKS

Dataset	Method	Accuracy (%)			Precision (%)			Recall (%)			F1-Score (%)		
		No Atk	FGSM	PGD	No Atk	FGSM	PGD	No Atk	FGSM	PGD	No Atk	FGSM	PGD
MedBioT	XGBoost	93.45	73.21	69.42	92.25	70.17	66.29	92.65	72.15	68.03	92.45	71.15	67.15
	E-GraphSAGE	92.95	76.83	72.50	91.65	74.31	69.87	92.05	76.11	71.35	91.85	75.20	70.60
	Anomal-E	92.05	74.98	71.15	90.85	71.62	68.08	91.25	73.70	69.94	91.05	72.65	69.00
	<i>HierGNN</i>	98.10	97.37	96.55	97.85	97.12	96.32	98.00	97.23	96.40	97.92	97.17	96.36
ToN-IoT	XGBoost	88.75	63.80	60.90	88.05	61.36	58.94	88.35	64.50	61.80	88.20	62.90	60.35
	E-GraphSAGE	84.35	66.50	62.15	83.95	64.72	60.33	84.15	66.80	62.70	84.05	65.75	61.50
	Anomal-E	87.65	68.85	65.90	87.05	67.51	64.32	87.35	69.10	66.50	87.20	68.30	65.40
	<i>HierGNN</i>	95.30	94.45	93.80	95.20	94.29	93.56	95.20	94.53	93.98	95.20	94.41	93.77
CICIDS2017	XGBoost	91.20	70.80	66.15	90.20	67.53	62.63	90.60	69.70	64.81	90.40	68.60	63.70
	E-GraphSAGE	86.50	65.60	61.30	85.60	66.11	61.11	64.90	59.70	55.25	73.80	62.70	58.00
	Anomal-E	85.80	64.10	60.50	80.50	63.95	59.19	67.20	58.40	54.10	73.20	61.00	56.50
	<i>HierGNN</i>	95.10	94.31	93.53	95.05	94.16	93.31	94.95	94.24	93.49	95.00	94.20	93.40

candidate for network intrusion detection in environments where collecting and labeling large datasets is not feasible.

3) *RQ3: Resilience Against Adversarial Attacks:* Answering RQ3 required evaluating *HierGNN*'s resilience to common adversarial evasion attacks. As glass box attacks represent a worst-case scenario where the attacker has complete knowledge of the model, a model that is robust to them is generally expected to be resilient against less-informed closed box attacks. We subjected the best-performing models to two well-known gradient-based glass box attacks: the Fast Gradient Sign Method (FGSM) and the more potent Projected Gradient Descent (PGD) attack. The evaluation was conducted on all three datasets, with models trained on 1000 labeled the data to provide a realistic baseline. Table III shows model performance before and after being subjected to these attacks.

The results clearly indicate that adversarial attacks significantly degrade the performance of all baseline models, whereas *HierGNN* exhibits remarkable resilience.

On the **MedBioT dataset**, under the FGSM attack, the F1-Score of XGBoost plummets from 92.45% to 71.15%, a drop

of 21.3 percentage points. The PGD attack causes an even more severe drop to 67.15%. In contrast, *HierGNN*'s F1-Score only decreases from 97.92% to 97.17% under FGSM and to 96.36% under PGD, demonstrating a minimal performance loss and superior stability.

A similar pattern is observed on the **ToN-IoT dataset**. XGBoost's accuracy drops from 88.75% to 63.80% (FGSM) and 60.90% (PGD). Other models like E-GraphSAGE and Anomal-E also suffer substantial performance degradation. *HierGNN*, however, shows only a minor dip in accuracy, from 95.30% to 94.45% (FGSM) and 93.80% (PGD). Its F1-score remains impressively high, above 93.70% even under the stronger PGD attack.

The resilience of *HierGNN* is further highlighted on the challenging **CICIDS2017 dataset**. The baseline models are highly vulnerable; for instance, the F1-score of E-GraphSAGE drops from 73.80% to 62.70% (FGSM) and 58.00% (PGD). *HierGNN*, on the other hand, maintains a very high F1-score, which only slightly reduces from 95.00% to 94.20% (FGSM) and 93.40% (PGD).

TABLE IV
COMBINED ABLATION STUDY ON MedBIoT, ToN-IoT, AND CICIDS2017, COMPARING MODELS TRAINED WITH 1000 RECORDS VERSUS 10% OF THE TRAINING DATA. PERFORMANCE IS EVALUATED UNDER NORMAL (NO ATTACK) AND ADVERSARIAL (PGD ATTACK) CONDITIONS

Dataset	Model Variant	Accuracy (%)				Precision (%)				Recall (%)				F1-Score (%)			
		1000 Records		10% Data		1000 Records		10% Data		1000 Records		10% Data		1000 Records		10% Data	
		No Atk	PGD	No Atk	PGD	No Atk	PGD	No Atk	PGD	No Atk	PGD	No Atk	PGD	No Atk	PGD	No Atk	PGD
MedBIoT	<i>HierGNN</i> (Full)	98.10	96.55	99.15	97.85	97.85	96.32	98.90	97.60	98.00	96.40	99.05	97.75	97.92	96.36	98.97	97.67
	w/o Packet-GAT	94.60	75.25	96.00	77.35	94.45	72.60	95.70	75.80	93.70	74.30	95.30	76.60	94.07	73.44	95.50	76.20
	w/o Flow-TGN	96.05	87.90	97.40	89.10	94.85	86.80	97.00	88.22	95.25	87.40	97.20	88.70	95.05	87.10	97.10	88.46
ToN-IoT	<i>HierGNN</i> (Full)	95.30	93.80	96.10	95.65	95.20	93.56	96.00	95.55	95.20	93.98	96.10	95.65	95.20	93.77	96.05	95.60
	w/o Packet-GAT	91.15	69.95	92.20	71.10	90.78	68.31	91.90	70.40	91.22	69.70	92.30	70.85	91.00	69.00	92.10	70.62
	w/o Flow-TGN	91.40	82.87	92.50	84.17	91.05	81.73	92.30	83.53	91.38	82.54	92.45	83.94	91.21	82.13	92.37	83.73
CICIDS2017	<i>HierGNN</i> (Full)	95.10	93.53	98.25	97.75	95.05	93.31	98.10	97.60	94.95	93.49	98.20	97.70	95.00	93.40	98.15	97.65
	w/o Packet-GAT	92.14	72.02	94.84	74.22	92.19	70.37	94.51	73.40	91.85	72.25	94.45	74.35	92.01	71.30	94.48	73.87
	w/o Flow-TGN	93.91	84.32	96.11	86.52	93.33	83.10	95.73	85.80	93.72	84.51	95.92	86.81	93.53	83.80	95.82	86.30

The findings for RQ3 demonstrate that *HierGNN* possesses a high degree of resilience against adversarial evasion attacks. While state-of-the-art methods experience a severe decline in detection performance, *HierGNN* maintains its high accuracy and F1-score with only a negligible drop. This robustness makes *HierGNN* a much more reliable and secure solution for deployment in real-world network environments where adversarial threats are a significant concern.

4) *RQ4: Architectural Contribution (Ablation Study)*: To address RQ4, we conducted an ablation study to quantify the contribution of *HierGNN*'s core architectural components: the packet-level adaptive filtering GNN (Packet-GAT) and the flow-level temporal GNN (Flow-TGN). We created two variants of our model: *HierGNN* w/o Packet-GAT (which uses mean aggregation instead) and *HierGNN* w/o Flow-TGN (which uses a simple MLP classifier on aggregated packet features). We compared these variants against the full *HierGNN* model on all three datasets, training with 1000 and 10% labeled data. We evaluated performance under both normal conditions and adversarial (PGD) attacks to isolate the components responsible for performance and resilience. The results are summarized in Table IV.

The ablation study reveals the individual contributions of *HierGNN*'s core components. Removing either the packet-level or flow-level component leads to a noticeable drop in performance, with the effect being most pronounced under adversarial attack.

Contribution of Packet-level GAT: The 'w/o Packet-GAT' variant shows the most significant performance degradation across all scenarios. On the MedBIoT dataset (1000 records), its F1-Score under PGD attack drops to 73.44%, a staggering 22.92 percentage points below the full model. This demonstrates that the packet-level adaptive filtering is critical for learning robust, fine-grained features that are resilient to adversarial perturbations. Without it, the model becomes highly vulnerable, and its data efficiency is compromised, as seen in the performance drops in both the 1000-record and 10%-data settings.

Contribution of Flow-level TGN: The 'w/o Flow-TGN' variant also underperforms compared to the full model, though the drop is less severe than removing the packet-level GAT. For instance, on the CICIDS2017 dataset (10% data), its F1-score under PGD attack is 86.30%, which is 11.35 points lower than the full model's 97.65%. This indicates that capturing

the temporal evolution of network flows is crucial for accurate and stable detection. The flow-level TGN effectively models the sequential nature of traffic, and its absence weakens the model's ability to understand the broader context of network behavior over time.

The ablation study confirms that both the packet-level adaptive filtering (GAT) and the flow-level temporal GNN (TGN) are indispensable components of the *HierGNN* architecture, with the packet-level GAT being particularly critical for adversarial resilience. Its superior contribution stems from its position as the first line of defense against feature perturbations. Gradient-based attacks like FGSM and PGD operate by introducing subtle, malicious noise to the input features, and the packet-level GAT is specifically designed to identify and suppress these manipulations at the most granular level. By filtering adversarial noise within each individual packet, it prevents corrupt data from propagating through the model. While the flow-level TGN is crucial for modeling temporal patterns, its effectiveness depends on the quality of the initial packet embeddings it receives. If these embeddings are compromised, the TGN's ability to classify correctly is severely hindered. Therefore, the packet-level module's role as a robust feature purifier is the primary reason for the model's overall resilience, ensuring that subsequent layers operate on a clean and reliable representation of the network traffic. The synergy between this robust packet-level analysis and the high-level temporal modeling is the key to *HierGNN*'s superior performance, validating our hierarchical design.

C. RQ5: Computational Performance Analysis

To evaluate the feasibility of deploying *HierGNN* in real-world edge environments, we analyzed its computational complexity, memory footprint, and inference latency. We measured the performance for processing a single network flow, averaged over 10,000 samples from the CICIDS2017 test set. The results are presented in Table V.

This result indicates that *HierGNN* is highly efficient, particularly when accelerated with a GPU, processing a flow in under 3 milliseconds. Even on a CPU, the latency remains low enough for real-time detection scenarios. The model's size is a modest 15.2 MB, and its memory footprint is well within the capabilities of contemporary edge devices. While more resource-intensive than a simple Decision Tree, *HierGNN*'s performance profile demonstrates its practicality

TABLE V
COMPUTATIONAL PERFORMANCE OF *HierGNN*

Metric	GPU	CPU
Inference Latency (ms/flow)	2.87	11.43
Memory Footprint (MB)	245	188
Model Size (MB)	15.2	
FLOPs (GFLOPs/flow)	0.89	

for deployment at the network edge, where its high accuracy and resilience are most needed.

V. CONCLUSION

In this article, we introduced *HierGNN*, a hierarchical graph neural network designed to address the critical challenge of resilient intrusion detection in consumer IoT environments with limited labeled data. Our comprehensive evaluation demonstrates that *HierGNN* not only establishes a new standard for detection accuracy but also excels in data efficiency and adversarial resilience. By hierarchically modeling network traffic from the packet to the flow level, *HierGNN* maintains high performance even under extreme data scarcity, achieving an F1-score over 0.96 with just 0.1% of labeled data. Furthermore, its architecture provides inherent resilience to adversarial attacks, showing only minimal performance degradation against FGSM and PGD attacks. Our ablation studies confirmed the vital contributions of both the packet-level adaptive filtering and the flow-level temporal GNN to this success.

Future work will proceed in several promising directions. First, we plan to evaluate the model's resilience against closed box, query-based adversarial attacks and explore the integration of specific defense mechanisms to counter them. Next, we aim to develop methods to enhance the explainability of *HierGANN*'s decisions, providing security analysts with insights into why a particular flow was flagged as malicious. Finally, we will assess the model's performance and scalability in more complex, high-volume network environments beyond consumer IoT, such as enterprise or carrier-grade networks.

REFERENCES

- [1] P. K. Sadhu, V. P. Yanambaka, and A. Abdelgawad, "Internet of Things: Security and solutions survey," *Sensors*, vol. 22, no. 19, p. 8227, 2022.
- [2] I. Lee and K. Lee, "The Internet of Things (IoT): Applications, investments, and challenges for enterprises," *Bus. Horizons*, vol. 58, no. 4, pp. 431–440, 2015.
- [3] S. Sicari, A. Rizzardi, L. A. Grieco, and A. Coen-Porisini, "Security, privacy and trust in Internet of Things: The road ahead," *Comput. Netw.*, vol. 76, pp. 146–164, Jan. 2015.
- [4] M. Antonakakis et al., "Understanding the mirai botnet," in *Proc. 26th USENIX Secur. Symp. (USENIX Secur.)*, 2017, pp. 1093–1110.
- [5] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017.
- [6] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Macia-Fernandez, and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Comput. Secur.*, vol. 28, nos. 1–2, pp. 18–28, 2009.
- [7] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.
- [8] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, pp. 1–11.
- [9] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models that are robust to adversarial attacks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018, pp. 1–28.
- [10] S. Ennaji, F. De Gaspary, D. Hitaj, A. Kbid, and L. V. Mancini, "Adversarial challenges in network intrusion detection systems: Research insights and future prospects," 2024, *arXiv:2409.18736*.
- [11] J. Zhou et al., "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, Apr. 2021.
- [12] J. Liu and M. Guo, "DIGNN-A: Real-time network intrusion detection with integrated neural networks based on dynamic graph," *Comput., Mater. Continua*, vol. 82, no. 1, pp. 817–842, 2025.
- [13] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, 2017, pp. 712–717.
- [14] F. Jiang et al., "Deep learning based multi-channel intelligent attack detection for data security," *IEEE Trans. Sustain. Comput.*, vol. 5, no. 2, pp. 204–212, Apr.–Jun. 2020.
- [15] M. M. Hassan, A. Gumaie, A. Alsanad, M. Alrubaihan, and G. Fortino, "A hybrid deep learning model for efficient intrusion detection in big data environment," *Inf. Sci.*, vol. 513, pp. 386–396, Mar. 2020.
- [16] N. Moustafa, N. Koroniotis, M. Keshk, A. Y. Zomaya, and Z. Tari, "Explainable intrusion detection for cyber defences in the Internet of Things: Opportunities and solutions," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 3, pp. 1775–1807, 3rd Quart., 2023.
- [17] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018, pp. 1–12.
- [18] K. Simonyan, A. Vedaldi, and A. Zisserman, "Deep inside convolutional networks: Visualising image classification models and saliency maps," in *Proc. 2nd Int. Conf. Learn. Represent. (ICLR)*, 2014, pp. 1–8.
- [19] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 3319–3328.
- [20] C. Do Xuan, M. H. Dao, and H. D. Nguyen, "APT attack detection based on flow network analysis techniques using deep learning," *J. Intell. Fuzzy Syst.*, vol. 39, no. 3, pp. 4785–4801, 2020.
- [21] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 6000–6010.
- [22] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2018, pp. 1–8.
- [23] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustain. Cities Soc.*, vol. 72, Sep. 2021, Art. no. 102994.
- [24] M. Saied, S. Guirguis, and M. Madbouly, "Review of filtering based feature selection for botnet detection in the Internet of Things," *Artif. Intell. Rev.*, vol. 58, no. 4, p. 119, 2025.
- [25] M. Eskandari, Z. H. Janjua, M. Vecchio, and F. Antonelli, "Passban IDS: An intelligent anomaly-based intrusion detection system for IoT edge devices," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 6882–6897, Aug. 2020.
- [26] C. Wang, P. Zheng, J. Gui, C. Hua, and W. U. Hassan, "Are we there yet? Unraveling the state-of-the-art graph network intrusion detection systems," 2025, *arXiv:2503.20281*.
- [27] Z. Xu et al., "Deep learning-based intrusion detection systems: A survey," 2025, *arXiv:2504.07839*.
- [28] R. Xu, G. Wu, W. Wang, X. Gao, A. He, and Z. Zhang, "Applying self-supervised learning to network intrusion detection for network flows with graph neural network," *Comput. Netw.*, vol. 248, Jun. 2024, Art. no. 110495.
- [29] J. Gu and S. Lu, "An effective intrusion detection approach using SVM with Naïve Bayes feature embedding," *Comput. Secur.*, vol. 103, Apr. 2021, Art. no. 102158.
- [30] B. Mahbooba, M. Timilsina, R. Sahal, and M. Serrano, "Explainable artificial intelligence (XAI) to enhance trust management in intrusion detection systems using decision tree model," *Complexity*, vol. 2021, pp. 1–11, Jan. 2021.
- [31] Q. Mao et al., "FeCoGraph: Label-aware federated graph contrastive learning for few-shot network intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 2266–2280, 2025.
- [32] P. Wu and H. Guo, "LuNet: A deep neural network for network intrusion detection," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, 2019, pp. 617–624.
- [33] A. Z. Alalmaie, P. Nanda, and X. He, "Zero trust-NIDS: Extended multi-view approach for network trace anonymization and auto-encoder CNN for network intrusion detection," in *Proc. IEEE Int. Conf. Trust, Security Privacy Comput. Commun. (TrustCom)*, 2022, pp. 449–456.

- [34] X. Ma et al., "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12012–12038, Dec. 2023.
- [35] J. Zhou, Z. Xu, A. M. Rush, and M. Yu, "Automating botnet detection with graph neural networks," 2020, *arXiv:2003.06344*.
- [36] W. W. Lo, G. Kulatilleke, M. Sarhan, S. Layeghy, and M. Portmann, "XG-BOT: An explainable deep graph neural network for botnet detection and forensics," *Internet Things*, vol. 22, Jul. 2023, Art. no. 100747.
- [37] J. Zheng and D. Li, "GCN-TC: Combining trace graph with statistical features for network traffic classification," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2019, pp. 1–6.
- [38] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Netw. Oper. Manag. Symp. (NOMS)*, Budapest, Hungary, 2022, pp. 1–9.
- [39] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, "Anomal-E: A self-supervised network intrusion detection system based on graph neural networks," *Knowl. Based Syst.*, vol. 258, Dec. 2022, Art. no. 110030.
- [40] H. Nguyen and R. Kashef, "TS-IDS: Traffic-aware self-supervised learning for IoT network intrusion detection," *Knowl. Based Syst.*, vol. 279, Nov. 2023, Art. no. 110966.
- [41] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security Privacy (SP)*, 2010, pp. 305–316.
- [42] Y. A. Farrukh, S. Wali, I. Khan, and N. Bastian, "XG-NID: Dual-modality network intrusion detection using a heterogeneous graph neural network and large language model," 2024, *arXiv:2408.16021*.
- [43] H. Li and D. Chasaki, "Heterogeneous GNN with express edges for intrusion detection in cyber-physical systems," in *Proc. Int. Conf. Comput., Netw. Commun. (ICNC)*, 2024, pp. 523–529.
- [44] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1126–1135.
- [45] A. Guerra-Manzanares, J. Medina-Galindo, H. Bahsi, and S. Nömm, "MedBIoT: Generation of an IoT botnet dataset in a medium-sized IoT network," in *Proc. Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2020, pp. 207–218.